

Logistic Regression

Outline

Introduction and Theory

Types and Assumptions

Mathematical Example

Implementation

Model Evaluation

Regularization and Feature Importance

Advanced Topics

Tools and Best Practices

Introduction and Theory

What is Logistic Regression?

- A statistical model for binary classification.
- Predicts the probability of class membership.
- Output: probability, mapped to classes using sigmoid.

Logistic vs Linear Regression

- **Linear Regression:** Predicts continuous outcomes.
- **Logistic Regression:** Predicts probability of class membership.
- Uses the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$$

Types and Assumptions

Types of Logistic Regression

- **Binary:** Two classes (e.g., spam or not spam).
- **Multinomial:** Three or more unordered classes.
- **Ordinal:** Ordered categorical outcomes.

Assumptions

- Binary dependent variable.
- Logit is linear in predictors.
- No multicollinearity.
- Observations are independent.
- Large sample size preferred.

Mathematical Example

Example Dataset: Fruits

Fruit	Weight (g)	Color Score	Apple? (y)
1	150	0.9	1
2	180	0.8	1
3	120	0.3	0
4	130	0.4	0

Model Equation

We model the probability of a fruit being an apple using:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- x_1 : Weight (in grams)
- x_2 : Color score (0 = green, 1 = red)

Given Coefficients and New Input

- Coefficients:

$$\beta_0 = -20, \quad \beta_1 = 0.1, \quad \beta_2 = 5$$

- New fruit input:

$$x_1 = 160, \quad x_2 = 0.85$$

Step 1: Compute Linear Combination

$$z = -20 + 0.1 \cdot 160 + 5 \cdot 0.85 = -20 + 16 + 4.25 = 0.25$$

Step 2: Apply Sigmoid Function

$$\hat{y} = \frac{1}{1 + e^{-0.25}} \approx \frac{1}{1 + 0.7788} \approx 0.562$$

- The model predicts a probability of $\hat{y} = 0.562$.
- With a threshold of 0.5, we classify the fruit as:

Apple (1)

Implementation

- Uses scikit-learn's LogisticRegression.

Example

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```


Key Hyperparameters

- `penalty`: L1, L2, elasticnet, or none.
- `C`: Inverse of regularization strength.
- `solver`: liblinear, saga, lbfgs, etc.
- `class_weight`: Deal with imbalance.

Model Evaluation

- Accuracy, Precision, Recall, F1 Score.
- Confusion Matrix.
- ROC AUC, Log Loss, Brier Score.

- Use `calibration_curve` or `CalibratedClassifierCV`.
- Important in risk-sensitive applications.

Regularization and Feature Importance

Regularization Techniques

- **L1 (Lasso)**: Feature selection.
- **L2 (Ridge)**: Shrinks large coefficients.
- **ElasticNet**: Mix of both.

Interpreting Coefficients

- Coefficients represent change in log-odds.
- Odds Ratio: $OR = e^{\beta}$
- Standardize features before interpretation.

Advanced Topics

Multiclass and Imbalanced Data

- Use `multi_class='multinomial'` for >2 classes.
- Handle imbalance with:
 - `class_weight='balanced'`
 - SMOTE / undersampling

Diagnostics and Feature Selection

- Use VIF to detect multicollinearity.
- Recursive Feature Elimination (RFE).
- Hosmer-Lemeshow test for goodness of fit.

Tools and Best Practices

- **Modeling:** `sklearn.linear_model`
- **Metrics:** `sklearn.metrics`
- **Visualization:** `seaborn`, `matplotlib`, `yellowbrick`
- **Feature Selection:** `SelectFromModel`, `RFE`

Best Practices

- Scale features for regularization.
- Handle class imbalance carefully.
- Evaluate both class labels and probabilities.
- Validate with cross-validation.
- Understand your coefficients.

Thank you!